CS1010J Lecture #8

2D Arrays

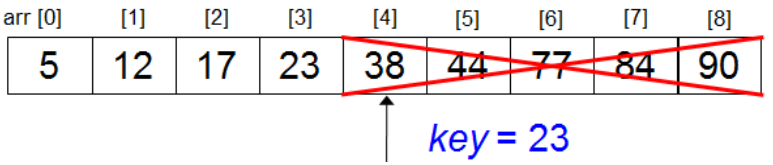
The matrix reloaded

NUS National University of Singapore
Department of Computer Science
School of Computing

Searching

PREVIOUS LECTURE

- Search key
  - The item you are looking for in an array
- Linear search
  - Search the array from one end to the other.
- Binary search
  - Compare arr[mid] with key
  - Each time reduce the scope of search by half.



- Binary search greatly outperforms linear search.

Selection Sort

- Scan the “alive” array to find the minimum item.
- Swap it with the first item of the “alive” array.

29	**10**	14	37	13
10	**29**	14	37	**13**
10	13	**14**	37	29
10	13	14	**37**	**29**
10	13	14	29	37

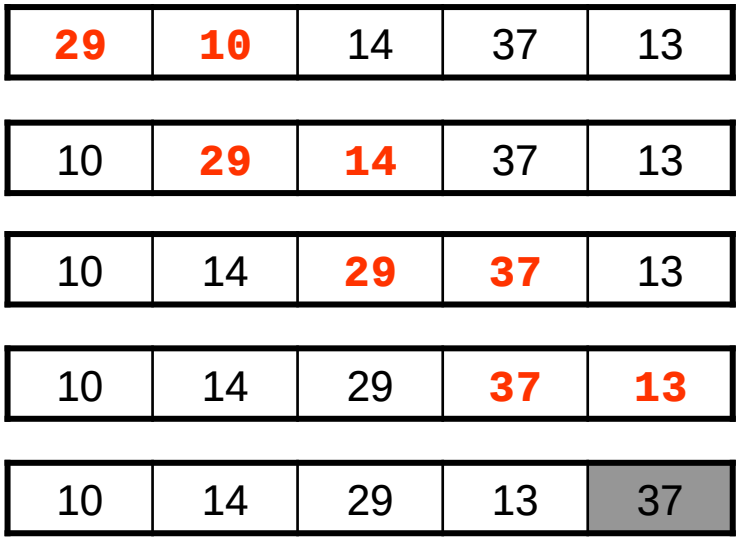
10 is the smallest, swap it with the first one, i.e. **29**.

sorted!

Bubble Sort

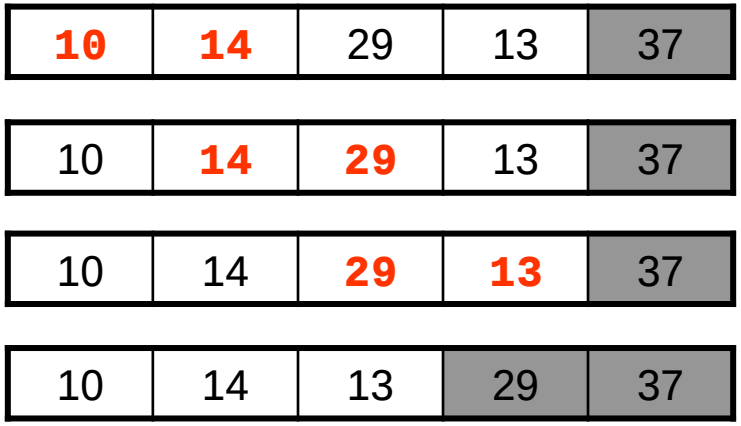
PREVIOUS LECTURE

- Move the largest item to the end of the array in each iteration by examining neighbours and swap them if they are out of order.



Pass 1

At the end of the pass 1, the largest item 37 is at the end.



Pass 2

At the end of the pass 2, the largest item 29 is at the end of the “alive” array.

Quiz

(CS1101C AY2007/08 Semester 2 Exam, Q12)

- Selection sort

	79	67	35	**22**	73
1st pass:	22	**67**	**35**	79	73
2nd pass:	22	35	**67**	79	73
3rd pass:	22	35	67	**79**	**73**
4th pass:	22	35	67	73	79

Quiz

(CS1101C AY2007/08 Semester 2 Exam, Q12)

- Enhanced bubble sort

	79	35	22	67	73
1st pass:	35	22	67	73	79
2nd pass:	22	35	67	73	79
3rd pass:	22	35	67	73	79

Learning Objectives

- At the end of this lecture, you should understand:
  - The concept of two-dimensional arrays.
  - How to create and use 2D arrays.
  - When to use arrays.

Two dimensional Arrays (1/2)

- In Java, we may declare a multi-dimensional array by using two or more sets of brackets.
- We only study two-dimensional arrays (2D arrays) in this course, which are useful in representing tabular information.
- Example:

Times Table - 10x10

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
1	**1**	2	3	4	5	6	7	8	9	10
2	2	**4**	6	8	10	12	14	16	18	20
3	3	6	**9**	12	15	18	21	24	27	30
4	4	8	12	**16**	20	24	28	32	36	40
5	5	10	15	20	**25**	30	35	40	45	50
6	6	12	18	24	30	**36**	42	48	54	60
7	7	14	21	28	35	42	**49**	56	63	70
8	8	16	24	32	40	48	56	**64**	72	80
9	9	18	27	36	45	54	63	72	**81**	90
10	10	20	30	40	50	60	70	80	90	**100**

Two dimensional Arrays (2/2)

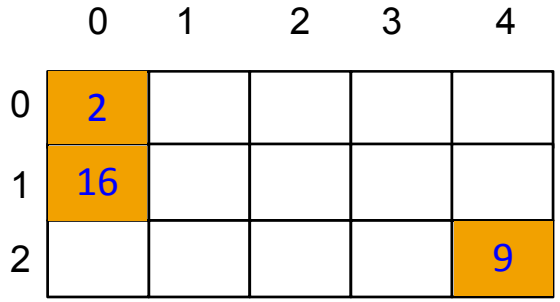
SYNTAX

<data type>[][] <variable> = new <data type>[<rows>] [<cols>];

- Example:

```
// array with 3 rows, 5 columns
int[][] a = new int[3][5];
a[0][0] = 2;
a[2][4] = 9;
a[1][0] = a[2][4] + 7;
```

Row number *Column number*



Lengths in 2D Arrays

- In Java, a 2D array is actually an array of arrays where each row is an array.
  - We will focus on 2D arrays in which each row has the same number of elements (i.e. matrix).
- Example:

Initialize a 2D array row by row

```
int[][] arr = {
      { 1, 2, 3, 4 },
      { 5, 6, 7, 8 },
      { 0, 0, 0, 0 }
};
System.out.println(arr.length);     // number of rows
System.out.println(arr[0].length); // length of 1st row
```

```
3
4
```

Passing 2D Arrays to Method Calls

```
class Sum2DArray {

  public static void main(String[] args) {
    int[][] foo = { { 1, 2, 3}, { 4, 5, 6 } };
    System.out.println("Sum = " + sumArray(foo) );
  }


  // Sum up elements in arr
  public static int sumArray( int[][] arr ) {

    int total = 0;
    for (int row = 0; row < arr.length; row++) {
      for (int col = 0; col < arr[row].length; col++) {
        total += arr[row][col];
      }
    }
    return total;
  }
}
```

Caller just pass array by name

*Give **the same array** a new name*

Q: What is the output?

Sum = 21

Demo #1 : Matrix Addition (1/2)

- A two-dimensional array where all rows have the same number of elements is also known as a matrix because it resembles that mathematical concept.
- To add two matrices, both must have the same size (i.e. same number of rows and columns).
- To compute C = A + B, where A, B, C are matrices:

$$c_{m,n} = a_{m,n} + b_{m,n}$$

- Example on 2×4 matrices:

$$\underbrace{\begin{pmatrix} 10 & 21 & 7 & 9 \\ 4 & 6 & 14 & 5 \end{pmatrix}}_{A} + \underbrace{\begin{pmatrix} 3 & 7 & 18 & 20 \\ 6 & 5 & 8 & 15 \end{pmatrix}}_{B} = \underbrace{\begin{pmatrix} 13 & 28 & 25 & 29 \\ 10 & 11 & 22 & 20 \end{pmatrix}}_{C}$$

Demo #1 : Matrix Addition (2/2)

```
// Sum up mtxA and mtxB to obtain mtxC and return it
public static int[][] sumMatrix(int[][] mtxA, int[][] mtxB) {

  int numRows = mtxA.length, numCols = mtxA[0].length;
  int[][] mtxC = new int[numRows][numCols];

  for (int row = 0; row < numRows; row++) {
    for (int col = 0; col < numCols; col++) {
      mtxC[row][col] = mtxA[row][col] + mtxB[row][col];
    }
  }
  return mtxC;
}
```

$$\begin{pmatrix} 10 & 21 & 7 & 9 \\ 4 & 6 & 14 & 5 \end{pmatrix} + \begin{pmatrix} 3 & 7 & 18 & 20 \\ 6 & 5 & 8 & 15 \end{pmatrix} = \begin{pmatrix} 13 & 28 & 25 & 29 \\ 10 & 11 & 22 & 20 \end{pmatrix}$$

A B C

Java Class : **Arrays**

Non-examinable

- Java provides several methods for performing common array manipulations in the java.util.Arrays class, e.g.:
  - Searching an array for a specific value to get the index at which it is placed (the binarySearch() method).
  - Comparing two arrays to determine if they have the same content (the equals() method).
  - Filling an array to place a specific value at each index (the fill() method).
  - Sorting an array into ascending order (the sort() method).
  - Returning a string representation of the contents of the specified array (the toString() method)

Drawback of Arrays

- Array has one major drawback: once created, the length of the array is fixed.

- To manipulate a bunch of data, array is
  - A **great** choice if operations are mostly search, retrieval or replacement.
  - A **poor** choice if operations such as insertion / deletion are most frequent.

- Java offers two "resizable" arrays: ArrayList and Vector to overcome the drawback of array.
  - Details will be covered in CS2030.

Today's Summary

Arrays II

Two dimensional arrays

- Syntax and usage

The pros and cons of using arrays